

## Assignment: Cursors & Triggers — Hands-on & Scenario-Based Practice

---

**Note:** Work inside `college_db.academics`. Write and test every statement in pgAdmin. For triggers, perform the DML that should fire them and verify the side effects.

### Setup Reference

Use the existing tables. You may need to create a small audit table as part of the exercises.

#### Q1. Simple Cursor FOR Loop

Write an anonymous DO block that uses a cursor FOR loop to print (RAISE NOTICE) the `full_name` and `cgpa` of every student with `cgpa >= 7.0`, ordered by `cgpa DESC`.

-- Write your DO \$\$ ... \$\$ block here:

```
DO $$
DECLARE
    student_record RECORD;
BEGIN
    FOR student_record IN
        SELECT full_name, cgpa
        FROM academics.students
        WHERE cgpa >= 7.0
        ORDER BY cgpa DESC
    LOOP
        RAISE NOTICE 'Name: %, CGPA: %',
            student_record.full_name,
            student_record.cgpa;
    END LOOP;
END $$;
```

#### Q2. Parameterized Cursor

Write a DO block that declares a parameterized cursor accepting a `dept_id`. Open it for a specific department, loop through the students, and RAISE NOTICE each `full_name`. Close the cursor properly.

-- Write your DO block here:

```
DO $$
DECLARE
    student_record RECORD;

    student_cursor CURSOR (p_dept_id INT) FOR
        SELECT full_name
        FROM academics.students
        WHERE dept_id = p_dept_id;
BEGIN

    OPEN student_cursor(1);
```

```
LOOP
    FETCH student_cursor INTO student_record;

    EXIT WHEN NOT FOUND;

    RAISE NOTICE 'Student: %',
        student_record.full_name;
END LOOP;

CLOSE student_cursor;
END $$;
```

### Q3. Cursor with Conditional Logic

Using a cursor (or FOR loop), walk through all enrollments that have a grade. For each row call (or simulate) grade-to-points logic and RAISE NOTICE the student\_id, course\_id, grade, and points. Skip rows where grade is NULL.

-- Write your DO block here:

```
CREATE OR REPLACE FUNCTION academics.grade_to_points(p_grade TEXT)
RETURNS NUMERIC
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN CASE UPPER(p_grade)
        WHEN 'A' THEN 10
        WHEN 'B' THEN 8
        WHEN 'C' THEN 6
        WHEN 'D' THEN 4
        WHEN 'F' THEN 0
        ELSE NULL
    END;
END;
$$;

DO $$
DECLARE
    enrollment_record RECORD;
    points NUMERIC;
BEGIN
    FOR enrollment_record IN
        SELECT student_id, course_id, grade
        FROM academics.enrollments
        WHERE grade IS NOT NULL
    LOOP
```

```
points := academics.grade_to_points(enrollment_record.grade);
```

```
RAISE NOTICE
```

```
'Student: %, Course: %, Grade: %, Points: %',  
enrollment_record.student_id,  
enrollment_record.course_id,  
enrollment_record.grade,  
points;
```

```
END LOOP;
```

```
END $$;
```

## Q4. Audit Table + AFTER UPDATE Trigger

Create an audit table academics.cgpa\_audit (audit\_id SERIAL PK, student\_id INT, old\_cgpa NUMERIC, new\_cgpa NUMERIC, changed\_at TIMESTAMPTZ DEFAULT NOW()).

Then create a trigger function + AFTER UPDATE OF cgpa trigger on students that inserts a row into cgpa\_audit whenever cgpa actually changes.

Test by updating a student's cgpa and SELECTing from the audit table.

-- Write CREATE TABLE, FUNCTION, TRIGGER, and test statements here:

```
CREATE TABLE IF NOT EXISTS academics.cgpa_audit
```

```
(  
    audit_id SERIAL PRIMARY KEY,  
    student_id INT,  
    old_cgpa NUMERIC,  
    new_cgpa NUMERIC,  
    changed_at TIMESTAMPTZ DEFAULT NOW()  
);
```

```
CREATE OR REPLACE FUNCTION academics.audit_cgpa_change()
```

```
RETURNS TRIGGER
```

```
LANGUAGE plpgsql
```

```
AS $$
```

```
BEGIN
```

```
IF OLD.cgpa IS DISTINCT FROM NEW.cgpa THEN
```

```
INSERT INTO academics.cgpa_audit
```

```
(  
    student_id,  
    old_cgpa,  
    new_cgpa  
)
```

```
VALUES
```

```
(
    OLD.student_id,
    OLD.cgpa,
    NEW.cgpa
);

END IF;

RETURN NEW;
END;
$$;

CREATE OR REPLACE TRIGGER trg_cgpa_audit
AFTER UPDATE OF cgpa
ON academics.students
FOR EACH ROW
EXECUTE FUNCTION academics.audit_cgpa_change();
```

#### Q5. BEFORE INSERT Validation Trigger

Create a BEFORE INSERT OR UPDATE trigger on enrollments that:

- Sets enroll\_date to CURRENT\_DATE if it is NULL
- Raises an EXCEPTION if enroll\_date is in the future

Test both the auto-set path and the rejection path.

-- Write FUNCTION, TRIGGER, and test INSERTs here:

```
CREATE OR REPLACE FUNCTION academics.validate_enrollment_date()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN

    IF NEW.enroll_date IS NULL THEN
        NEW.enroll_date := CURRENT_DATE;
    END IF;

    IF NEW.enroll_date > CURRENT_DATE THEN
        RAISE EXCEPTION
            'Enrollment date cannot be in the future: %',
            NEW.enroll_date;
    END IF;

    RETURN NEW;
END;
$$;
```

```
CREATE OR REPLACE TRIGGER trg_validate_enrollment_date
BEFORE INSERT OR UPDATE
ON academics.enrollments
FOR EACH ROW
EXECUTE FUNCTION academics.validate_enrollment_date();
```

```
INSERT INTO academics.enrollments
(student_id, course_id, enroll_date, grade)
VALUES
(108, 201, NULL, 'B');
```

## Q6. Protect Parent Rows (BEFORE DELETE)

Create a BEFORE DELETE trigger on departments that prevents deletion if any students still reference that dept\_id. Raise a clear EXCEPTION naming the department. Test by attempting to delete a department that has students and one that does not (if available).

-- Write FUNCTION, TRIGGER, and test DELETes here:

```
CREATE OR REPLACE FUNCTION academics.prevent_department_delete()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    student_count INT;
BEGIN

    SELECT COUNT(*)
    INTO student_count
    FROM academics.students
    WHERE dept_id = OLD.dept_id;

    IF student_count > 0 THEN
        RAISE EXCEPTION
            'Cannot delete department "%" because students still reference it.',
            OLD.dept_name;
    END IF;

    RETURN OLD;
END;
$$;
```

```
CREATE OR REPLACE TRIGGER trg_prevent_department_delete
BEFORE DELETE
ON academics.departments
FOR EACH ROW
EXECUTE FUNCTION academics.prevent_department_delete();
```

```
DELETE FROM academics.departments
WHERE dept_id = 1;
```

## Q7. AFTER INSERT/DELETE Counter Trigger

Add a column enroll\_count INTEGER DEFAULT 0 to academics.courses (if it does not exist). Create an AFTER INSERT OR DELETE trigger on enrollments that keeps courses.enroll\_count in sync (increment on insert, decrement on delete, never below 0). Test by inserting and deleting enrollments and checking the count.

-- Write ALTER TABLE, FUNCTION, TRIGGER, and test statements here:

```
ALTER TABLE academics.courses
```

```
ADD COLUMN IF NOT EXISTS enroll_count INTEGER DEFAULT 0;
```

```
UPDATE academics.courses c
```

```
SET enroll_count =
```

```
(
```

```
    SELECT COUNT(*)
```

```
    FROM academics.enrollments e
```

```
    WHERE e.course_id = c.course_id
```

```
);
```

```
CREATE OR REPLACE FUNCTION academics.update_enroll_count()
```

```
RETURNS TRIGGER
```

```
LANGUAGE plpgsql
```

```
AS $$
```

```
BEGIN
```

```
    IF TG_OP = 'INSERT' THEN
```

```
        UPDATE academics.courses
```

```
        SET enroll_count = enroll_count + 1
```

```
        WHERE course_id = NEW.course_id;
```

```
    RETURN NEW;
```

```
    ELSIF TG_OP = 'DELETE' THEN
```

```
        UPDATE academics.courses
```

```
        SET enroll_count = GREATEST(enroll_count - 1, 0)
```

```
        WHERE course_id = OLD.course_id;
```

```
    RETURN OLD;
```

```
END IF;
```

END;

\$\$;

```
CREATE OR REPLACE TRIGGER trg_update_enroll_count
AFTER INSERT OR DELETE
ON academics.enrollments
FOR EACH ROW
EXECUTE FUNCTION academics.update_enroll_count();
```

## Q8. Trigger that Calls a Function

Using the `grade_to_points` (or similar) function from Day 7, create a BEFORE INSERT OR UPDATE trigger on enrollments that rejects any grade whose points would be NULL (i.e., invalid grade codes). Raise EXCEPTION 'Invalid grade: %'. Test with a valid and an invalid grade.

-- Write FUNCTION (if needed), TRIGGER, and test statements here:

## Q9. Multi-event Trigger with TG\_OP

Create a single trigger function that handles both UPDATE and DELETE on students and writes to an audit table with an action column ('UPDATE' or 'DELETE'). Use TG\_OP to decide what to log. Attach it as an AFTER trigger. Demonstrate both operations.

-- Write everything and test both UPDATE and DELETE here:

## Q10. Disable / Enable / Drop Trigger

- Disable one of the triggers you created.
- Perform a data change that would normally fire it and confirm it did not fire.
- Re-enable the trigger.
- Drop a trigger you no longer need.

-- Write ALTER TABLE ... DISABLE/ENABLE TRIGGER and DROP TRIGGER statements here:

## Q11. Cursor vs Set-based Comparison

- Write a cursor-based DO block that counts how many students have `cgpa > 8`.
- Write the equivalent set-based SELECT COUNT(\*).
- In 2-3 sentences explain when a cursor might still be justified despite set-based SQL usually being better.

-- (a), (b), and (c) here:

## Q12. Cleanup & Design Note

- List the triggers currently defined on your academics tables (query `information_schema` or `pg_trigger`).
- DROP any experimental triggers and audit tables you do not want to keep.
- In 2-3 sentences describe a real business rule in the college domain that is a good fit for a trigger (and why application code alone would be riskier).

-- (a) query/list, (b) DROP statements, (c) design note:

**Note:** Useful triggers can remain. Next we (Indexes, SQL Locking & Normalization) will focus on performance and design quality of the schema you have built across all eight days.